

Data Structures & Algorithms (CSC-214)

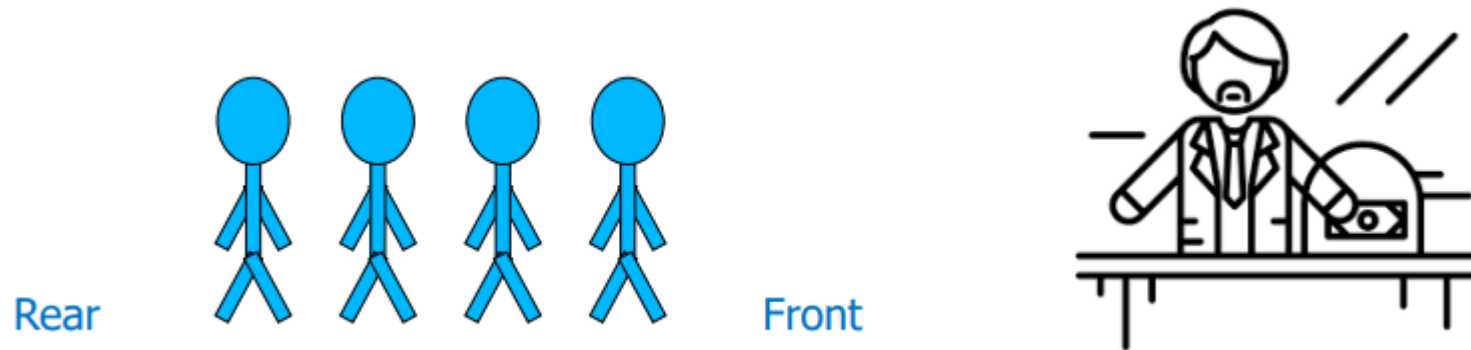
Queues – Array based implementation

Queues

- Queue is First-In-First-Out (FIFO) data structure
 - First element added to the queue will be first one to be removed
- Queue implements a special kind of list
 - Items are inserted at one end (the rear)
 - Items are deleted at the other end (the front)

Queue – Analogy (1)

- A queue is like a line of people waiting for a bank teller
- The queue has a front and a rear
 - New people must enter the queue at the rear
 - An item is always taken from the front of the queue



Queues – Examples

- Billing counter
 - Booking movie tickets
 - Queue for paying bills
- A print queue
- Vehicles on toll-tax bridge
- Luggage checking machine
- And others?

Queues – Applications

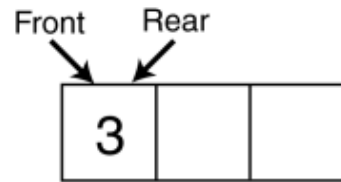
- Operating systems
 - Process scheduling in multiprogramming environment
 - Controlling provisioning of resources to multiple users (or processes)
- Communication software
 - Hold messages/packets in order of their arrival
 - Messages are usually transmitted faster than the time to process them
 - The most common application is in client-server models
 - Multiple clients may be requesting services from one or more servers
 - Some clients may have to wait while the servers are busy
 - Those clients are placed in a queue and serviced in the order of arrival

Basic Operations (Queue ADT)

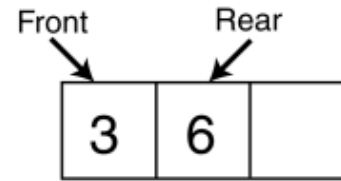
- **MAKENULL(Q)**
 - Makes Queue Q be an empty list
- **FRONT(Q)**
 - Returns the first element on Queue Q
- **ENQUEUE(x,Q)**
 - Inserts element x at the end of Queue Q
- **DEQUEUE(Q)**
 - Deletes the first element of Q
- **EMPTY(Q)**
 - Returns true if and only if Q is an empty queue

Enqueue And Dequeue Operations

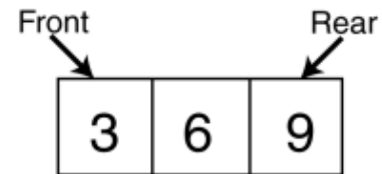
Enqueue(3);



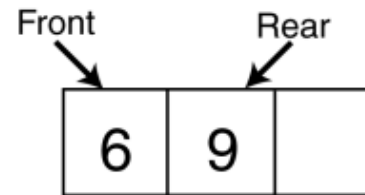
Enqueue(6);



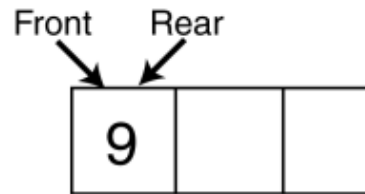
Enqueue(9);



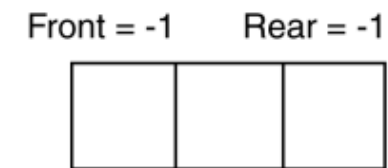
Dequeue();



Dequeue();



Dequeue();



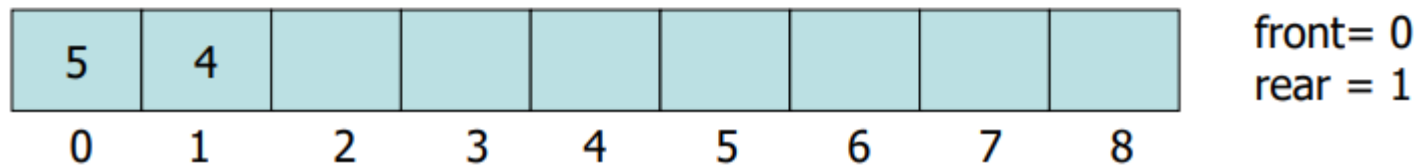
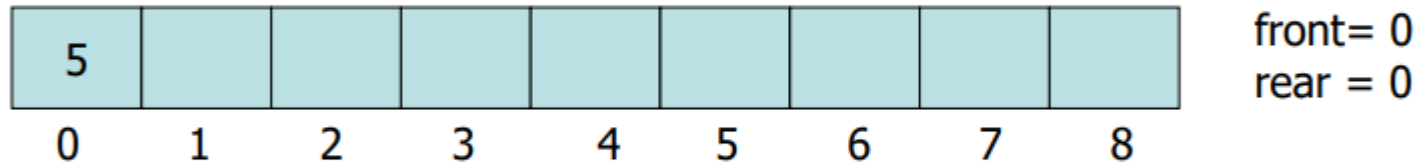
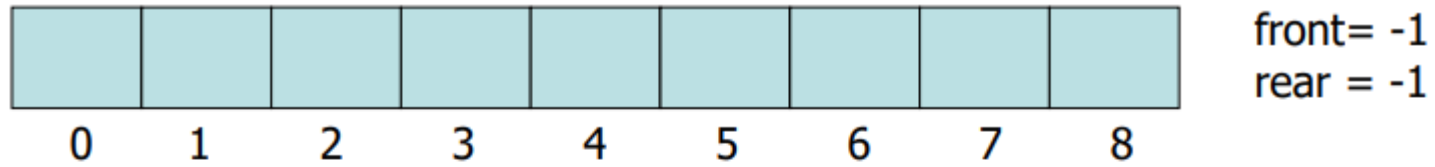
Implementation

- Static
 - Queue is implemented by an array
 - Size of queue remains fix
- Dynamic
 - A queue can be implemented as a linked list
 - Expand or shrink with each enqueue or dequeue operation

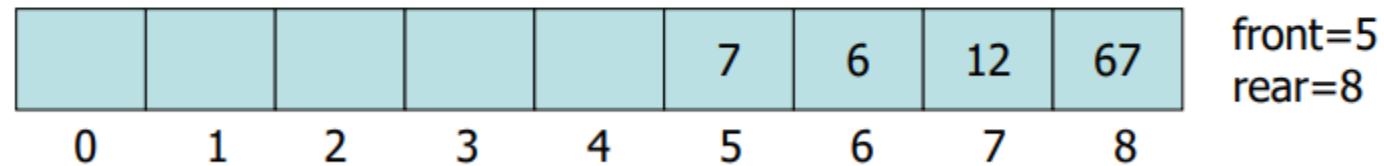
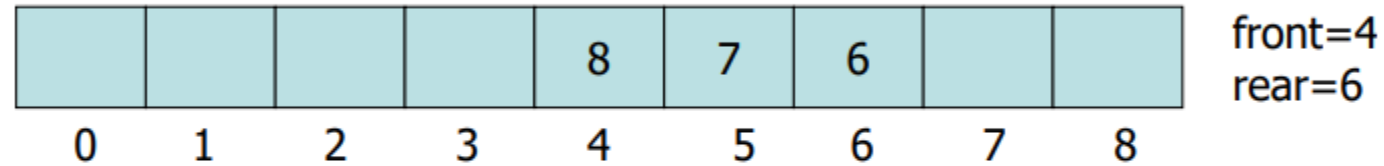
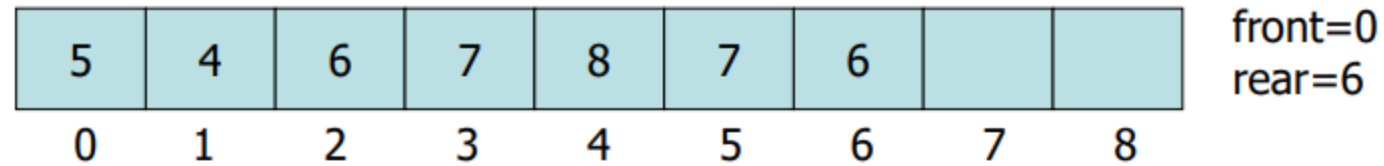
Array Implementation

- Use two counters that signify rear and front
- When queue is empty
 - Both front and rear are set to -1
- When there is only one value in the Queue
 - Both rear and front have same index
- While enqueueing increment rear by 1
- While dequeueing, increment front by 1

Array Implementation Example (1)



Array Implementation Example (2)



Problem: How can we insert more elements?
Rear index can not move beyond the last element....

Using Circular Queue

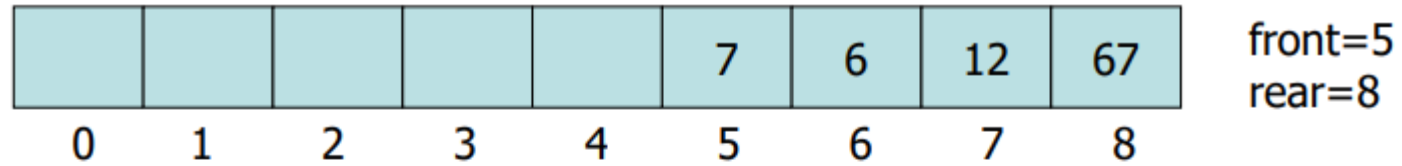
- Allow rear to wrap around the array

```
if(rear == queueSize-1)
    rear = 0;
else
    rear++;
```

- Alternatively, use modular arithmetic

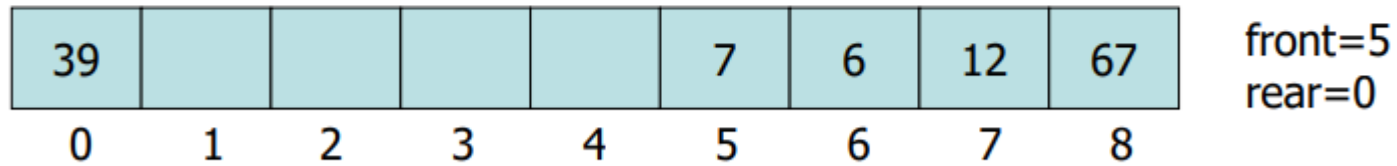
```
rear = (rear + 1) % queueSize;
```

Example



Enqueue 39

- $\text{Rear} = (\text{Rear} + 1) \bmod \text{queueSize} = (8 + 1) \bmod 9 = 0$



Problem: How to avoid overwriting an existing element?

How to Determine Empty and Full Queues?

- A counter indicating number of values/items in the queue
- Without using an additional counter (only relying on front and rear)

Array-based Implementation

Array Implementation – Code (1)

```
class IntQueue
{
private:
    int* queueArray; // Pointer to array implemented as Queue
    int queueSize; // Total size of the Queue
    int front;
    int rear;
    int numItems; // Number of items currently in the Queue
public:
    IntQueue(int);
    ~IntQueue();
    bool isEmpty();
    bool isFull();
    bool enqueue(int);
    bool dequeue(int&);
    void makeNull();
};
```


Array Implementation – Code (2)

```
IntQueue::IntQueue(int input)
{
    queueArray = new int[input]; // Pointer to array implemented as Queue
    queueSize = input; // Total size of the Queue
    front = -1;
    rear = -1;
    numItems = 0; // Number of items currently in the Queue
}
```

```
IntQueue::~~IntQueue()
{
    delete[] queueArray;
}
```

```
bool IntQueue::isEmpty()
{
    if (numItems == 0)
        return true;
    else
        return false;
}

bool IntQueue::isFull()
{
    if (numItems == queueSize)
        return true;
    else
        return false;
}

void IntQueue::makeNull()
{
    front = -1;
    rear = -1;
    numItems = 0;
}
```

Array Implementation – Code (3)

```
bool IntQueue::enqueue(int num)
{
    if (isFull())
    {
        cout << "Overflow.\n";
        return false;
    }
    // Calculate the new rear position
    rear = (rear + 1) % queueSize;
    // Insert new item
    queueArray[rear] = num;
    // Update item count
    numItems++;
    return true;
}
```

```
bool IntQueue::dequeue(int& num)
{
    if (isEmpty())
    {
        cout << "The queue is empty.\n";
        return false;
    }
    // Move front
    front = (front + 1) % queueSize;
    // Retrieve the front item
    num = queueArray[front];
    // Update item count
    numItems--;
    return true;
}
```

Using Queues

```
int main()
{
    IntQueue iQueue(5);
    cout << "Enqueuing 5 items...\n";
    // Enqueue 5 items.
    for (int x = 0; x < 5; x++)
        iQueue.enqueue(x);
    // Attempt to enqueue a 6th item.
    cout << "Now attempting to enqueue again...\n";
    iQueue.enqueue(5);
    // Dequeue and retrieve all items in the queue
    cout << "The values in the queue were:\n";
    while (!iQueue.isEmpty()) {
        int value;
        iQueue.dequeue(value);
        cout << value << endl;
    }
}
```

 Microsoft Visual Studio Debug Console

```
Enqueuing 5 items...
Now attempting to enqueue again...
Overflow.
The values in the queue were:
0
1
2
3
4
```

Alternative Array-based Implementation

Alternative Implementation – Code (1)

```
class CQueue
{
public:
    int* queueArray; // Pointer to array implemented as Queue
    int queueSize; // Total size of the Queue
    int front;
    int rear;

    CQueue(int size);
    ~CQueue();
    bool isFull();
    bool isEmpty();
    bool enqueue(int num);
    bool dequeue(int& num);
    void makeNull();
};
```

Alternative Implementation – Code (2)

```
bool CQueue::isEmpty()
{
    if (front == -1)
        return true; // we can check "rear" too
    else
        return false;
}

bool CQueue::isFull()
{
    if ((rear + 1) % queueSize == front)
        return true;
    else
        return false;
}
```

```
CQueue::CQueue(int size)
{
    front = rear = -1;
    queueArray = new int[size];
    queueSize = size;
}

CQueue::~CQueue() //destructor
{
    delete[] queueArray;
}

void CQueue::makeNull()
{
    front = -1;
    rear = -1;
}
```

Alternative Implementation – Code (3)

```
bool CQueue::enqueue(int num)
{
    if (isFull()) {
        cout << "Overflow\n";
        return false;
    }
    if (isEmpty())
        rear = front = 0;
    else
        rear = (rear + 1) % queueSize;
    queueArray[rear] = num;
    return true;
}
```

```
bool CQueue::dequeue(int& num)
{
    if (isEmpty()) {
        cout << "Underflow\n";
        return false;
    }
    num = queueArray[front];
    if (front == rear) //only one element in the queue
        front = rear = -1;
    else
        front = (front + 1) % queueSize;
    return true;
}
```

Comparison: enqueue Operation

```
bool CQueue ::enqueue(int num);
{
    if ( isFull() ) {
        cout<<"Overflow";
        return false;
    }
    if (isEmpty())
        rear = front = 0;
    else
        rear=(rear+1) % queueSize;

    queueArray[rear] = num;
    return true;
}
```

```
bool IntQueue::enqueue(int num)
{
    if (isFull())
    {
        cout << "Overflow.\n";
        return false;
    }
    // Calculate the new rear position
    rear = (rear + 1) % queueSize;
    // Insert new item
    queueArray[rear] = num;
    // Update item count
    numItems++;
    return true;
}
```


- Any Questions?